

Text Classification Pipeline Forensics

Diagnosing Trustworthy Change in a Disaster-Tweet Classifier

line 1: Mohamed Elshenawy
line 2: 50050354
line 3: me248@connect.hkust-gz.edu.cn

line 1: Shrook Mostafa
line 2: 50050445
line 3: smostafa920@connect.hkust-gz.edu.cn

line 1: Fatma AbdelMaksoud
line 2: 50050641
line 3: fa488@connect.hkust-gz.edu.cn

Abstract—We build a CPU-only TF-IDF + Logistic Regression pipeline for the Kaggle Disaster Tweets binary classification task and investigate five hypotheses about pipeline and data-quality changes using a fixed train/dev/hold-out split (4567/1523/1523 examples). Rather than reporting only score deltas, we pair every comparison with McNemar's exact test and, where relevant, a bootstrap confidence interval and a mechanism-level trace of why a score changed. Across all five investigations, we find a consistent pattern: several headline-looking improvements are not statistically distinguishable from their baseline at this sample size, one “safe-looking” data correction significantly hurts held-out performance, and a normalization choice's real cause is unrelated to its stated hypothesis. We argue this pattern of over-attributing small score changes to intended mechanisms is the central risk the assignment's forensics framing warns against, and we show a concrete, reproducible methodology for catching it.

Index Terms—text classification, TF-IDF, reproducibility, McNemar's test, data quality, disaster tweets

I. PROJECT PROBLEM AND GOAL

Text classification pipelines are frequently evaluated by a single held-out score, but a score change alone does not indicate why a model improved or whether the improvement reflects genuine task understanding, threshold artifacts, or dataset-specific shortcuts. The central problem addressed in this project is diagnosing whether five specific categories of pipeline change to a disaster-tweet classifier are genuinely trustworthy: (1) baseline reproducibility against a published reference contract, (2) text normalization choices, (3) reliance on shallow/metadata features as opposed to text content, (4) decision-rule and model choices, and (5) training-data quality issues. Our goal is not simply to maximize held-out F1, but to determine, for each change, whether the observed effect is statistically real, mechanistically matches its stated hypothesis, and would plausibly generalize beyond this specific fixed split.

II. METHODOLOGY

A. Data and split

We use the full labeled Kaggle Disaster Tweets train.csv (7613 rows; 3271 positive / 4342 negative), downloaded from the public mirror specified by the assignment. We apply the provided fixed, stratified split (split_indices.json, seed 3102) to obtain 4567 training, 1523 development, and 1523 held-out examples, and verify zero id overlap and zero duplicate ids across splits before any modeling. The development split is used exclusively for selecting preprocessing, thresholds, and model variants; the held-out split is touched exactly once per frozen decision.

B. Baseline model

Our reference-reproducing configuration is a TF-IDF vectorizer (unigrams only, min_df=1, max_df=1.0, no sublinear term-frequency scaling) feeding a Logistic Regression classifier (C=2.0, lbfgs solver, seed 3102), selected via a manual grid search targeting the published reference metric (reference_baseline_f1 = 0.7574, tolerance ± 0.001).

C. Statistical validation.

Because the held-out split contains only 1523 examples, raw F1 deltas of the size typically produced by a single lever change (0.001–0.02) are not, by themselves, evidence of a real effect. For every controlled comparison we therefore additionally report McNemar's exact test on the paired right/wrong outcomes of the two models being compared, and, where useful, a 2000-resample bootstrap 95% confidence interval.

III. MAIN EVIDENCE AND RESULTS

Table I summarizes the frozen decision and held-out result for each ticket.

TABLE I
SUMMARY OF FROZEN DECISIONS AND HELD-OUT RESULTS

Ticket	Decision	Dev F1	Held F1	Significance
1 – Baseline	Unigram TF-IDF, C=2.0	0.7514	0.7576*	ns (p=0.63) vs naive
2 – Normaliz.	Keep hashtag tokens	0.7518	0.7595	ns (p=0.25) 3/1523 flip
3 – Shortcut	N/A (diagnostic)	kw=0.660	kw=0.691 (91% of text)	ns (p=1.0) combined
4 – Decision	Threshold 0.45	0.7568	0.7666	ns (p=0.71); generalizes well
5 – Data qual.	Majority-vote fix, 12 rows	0.7545 (+0.0031)	0.7516 (−0.0060)	SIG, harmful (p=0.012)

*matches reference (0.7574) within ± 0.001 tolerance.

The overall pattern is notable: of five investigated levers, only one (Ticket 5's label fix) produced a statistically significant held-out effect, and that effect was negative relative to its hypothesis. The remaining four levers each produced small, plausible-looking F1 changes that did not clear the bar of paired statistical significance at this sample size.

IV. CASE ANALYSIS

A. Representative Case 1: A Normalization Fix With the Wrong Mechanism (Ticket 2)

The dev-selected normalization change keeping literal #hashtag tokens instead of unwrapping them to bare words improved held-out F1 from 0.7576 to 0.7595 by flipping exactly 3 of 1523 predictions. Inspecting those 3 examples

directly shows that 2 of the 3 flipped tweets contain no hashtag at all, and the third tokenizes identically under both configurations once passed through scikit-learn's default tokenizer (which already strips # regardless of preprocessing). Diffing the two configurations' fitted training vocabularies (both size 11,156, but composition differs by 6 words) traces the actual cause to a whitespace bug: unwrapping #hashtag deletes the # character without inserting a space, so a hashtag glued directly to a preceding word (e.g., "Rare#Deals_UK") collapses into one malformed token instead of remaining two valid words. The measured improvement is real but is not evidence for the stated hypothesis - it is evidence of a preprocessing implementation defect that a different fix (inserting a space on unwrap) would likely resolve equally well.

B. Representative Case 2: A "Safe" Data Fix That Significantly Backfires (Ticket 5)

Eleven exact-duplicate training texts (12 rows) carry conflicting labels. Relabeling each to its group's majority vote is a minimal, well-targeted correction with no apparent downside. Applying this fix and retraining the unchanged reference pipeline improves development F1 (0.7514 $\rightarrow$ 0.7545) but decreases held-out F1 (0.7576 $\rightarrow$ 0.7516), a drop confirmed statistically significant by McNemar's exact test (10 previously-correct held-out predictions become incorrect against only 1 improvement; $p = 0.012$). With only 12 of 4567 training rows affected, the retrained decision boundary shifts in a direction that happens to help the specific development examples without helping and here, actively hurting a disjoint held-out set. This case demonstrates that even a correction with strong face validity requires held-out verification before being trusted.

V. DIFFICULTIES AND SOLUTIONS

A. Difficulty 1: Naive baseline did not reproduce the reference

A first-pass baseline using common default settings missed the reference held-out F1 by -0.0134 , outside tolerance. We addressed this with a systematic manual probe over vectorizer and regularization settings, isolating each change before combining them.

B. Difficulty 2: Small held-out sample size made raw deltas unreliable

With only 1523 held-out examples, F1 changes of 0.001–0.02 are frequently within sampling noise. We addressed this by adding McNemar's exact paired test and bootstrap confidence intervals to every controlled comparison, rather than reporting F1 deltas alone.

C. Difficulty 3: A measured improvement's cause did not match its hypothesis.

The Ticket 2 result initially appeared to confirm that literal hashtag tokens carry useful signal. We addressed this by tracing flipped predictions back to raw text, checking actual tokenizer output, and diffing fitted vocabularies directly, which revealed the true mechanism rather than the hypothesized one.

VI. AI USAGE DECLARATION

AI assistance (Claude, ChatGPT, Gemini, Codex) was used throughout this project, primarily to draft the experiment scripts in `pipeline/` and `EXPERIMENTS/` — vectorizer/model setup, split-integrity checks, significance tests (McNemar, bootstrap CI), and result-export code were AI-drafted. Every quantitative claim was confirmed by independently re-running the corresponding script and checking the output against what is reported here; AI-suggested numbers were checked before being trusted, and at least one AI-drafted error count (Ticket 3/4 wrapup) was caught as incorrect and replaced with the real terminal output. Interpretive conclusions were also corrected by hand where an AI-drafted explanation didn't match the evidence - most notably Ticket 2, where an early AI-drafted claim that "hashtag tokens carry signal" was rejected after manually diffing tokenizer output and fitted vocabularies, which showed the real mechanism was a whitespace bug instead. Final judgment calls (which threshold to adopt, which data-quality dispositions to assign, whether a shortcut looked legitimate or artifact-driven) were made by the team based on that verification, not by AI.

Team members' individual AI-assistance entries are in `logs/chat.md`.

VII. DISCUSSION AND LIMITATIONS

Several limitations qualify these findings. First, all comparisons use a single fixed split and seed; we cannot distinguish "reproduces the reference" from "reproduces the reference on this particular split" without alternative splits. Second, our held-out sample ($n=1523$) limits statistical power: McNemar's test with roughly 30–40 discordant pairs cannot reliably detect effects smaller than several percentage points, so several "not significant" results should be read as "not detectably different at this sample size," not as proof of equivalence. Third, we did not correct for multiple comparisons across the several McNemar tests performed; this modestly raises the chance that at least one nominally significant result (Ticket 5) is a false positive, though the consistency of its direction and effect size argue against pure chance. Fourth, the Ticket 3 legitimacy judgment for shallow keyword signal was based on manual inspection of a small sample (5 of 21 near-pure keywords) rather than a systematic classification. Fifth, the Ticket 4 second-classifier comparison (linear SVM via SGD) used default optimizer settings without independent tuning, so its underperformance reflects our specific configuration rather than a general claim about linear SVMs for this task. Finally, this pipeline is CPU-only and restricted to a bag-of-words TF-IDF representation; we make no claims about how these findings would transfer to contextual embedding-based models.

VIII. CONCLUSION

Across five investigations into a disaster-tweet classification pipeline, we find that headline F1 changes frequently do not survive paired statistical testing at this dataset's held-out sample size, and that even when an effect is measurable, its underlying mechanism can differ from the hypothesis that

motivated the experiment. The one effect we found to be both statistically significant and directionally clear; a well-intentioned training-label correction was harmful rather than helpful on held-out data, despite improving development performance and despite being a minimal, defensible-looking fix. We conclude that evaluating a text classification pipeline change requires more than a single frozen-split F1 comparison: it requires paired significance testing, mechanism-level verification of why predictions changed, and skepticism toward development-split improvements that have not been confirmed on held-out data.

REFERENCES

- [1] Kaggle / CrowdFlower, “Real or Not? NLP with Disaster Tweets,” public mirror: ucbrise/kaggle-nlp-disasters, GitHub.
- [2] F. Pedregosa et al., “Scikit-learn: Machine Learning in Python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
- [3] Q. McNemar, “Note on the sampling error of the difference between correlated proportions or percentages,” *Psychometrika*, vol. 12, no. 2, pp. 153–157, 1947.